PI-NCA: Architectures and Results

Neural Cellular Automata, hybrids, and operator baselines for PDE emulation | branch research/jax-migration

A guide to every model in this study — what it is, how it works, and how it performed. Branch: `research/jax-migration`. Code: `src/pinca_jax/`.

All numbers here come from the committed benchmark files in `results/`. They were produced on a CPU host at reduced scale (small grids, short training), so treat the **ordering** as the finding and the exact digits as provisional.

---

1. The setup in one page

Every model in this study does the **same job**: look at the current state of a physical field, and predict what it looks like one time step later.

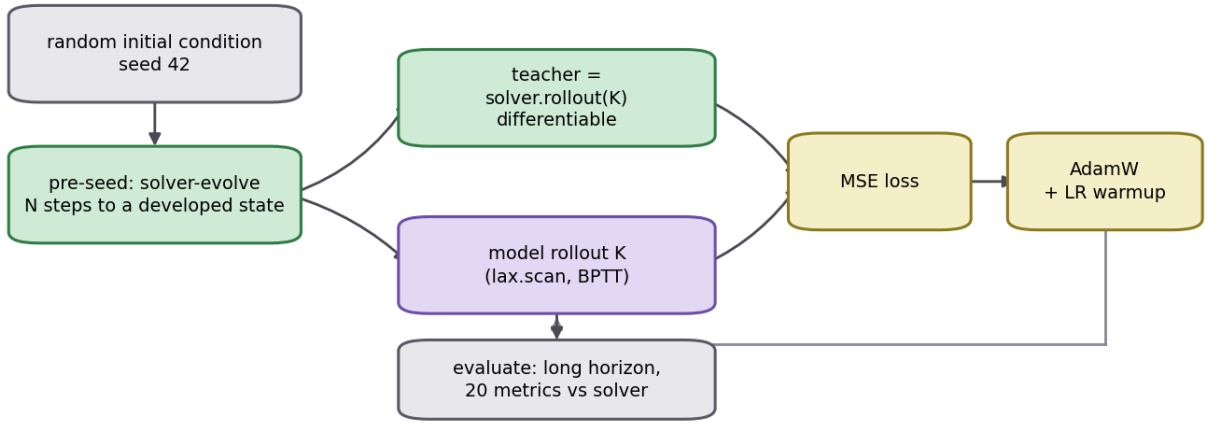
```
model:  state at time t   ->   state at time t+1
```

Apply it over and over and you get a simulation. This is called an **autoregressive emulator**.

To train it we use a **teacher**: a normal, hand-written PDE solver (`equations/pdes.py`). The teacher is correct but slow. We roll the teacher forward K steps, roll the model forward K steps from the same starting point, and train the model to match. This is distillation.

Because every architecture is trained against the same teacher, from the same starting conditions, over the same horizon, and scored with the same metrics, the comparison is fair. That shared pipeline is the whole reason the results mean anything.

H. Shared training pipeline - identical for every architecture



Every architecture trains the same way - same teacher, same horizon, same metrics - so the comparison is apples to apples.

Figure H — `docs/figures/arch/arch_training_pipeline.png`

Three details in that pipeline matter:

- **Pre-seeding.** Random initial conditions are unrealistically smooth. We run the solver

forward a few steps first, so the model trains on *developed* patterns, not just blobs.

- **Zero-initialised output head.** Every model starts as the identity map — it initially

predicts "nothing changes". Training then only has to learn the *change*. This is a much easier starting point than random output.

- **Training through the rollout (BPTT).** The loss is computed after K steps, not one, so

the model sees its own accumulated error during training. Ablation A6 shows this is decisive for unstable problems.

The physics problems (PDEs)

Ten in 2-D, six in 3-D. What matters for reading the results is which **structure** each one has, because that is what selects the winning architecture:

Property	Meaning	PDEs with it
Conservative	Total mass stays constant	heat, advection–diffusion, shallow-water, Cahn–Hilliard
Non-conservative	Source terms create/destroy quantity	Nagumo, FitzHugh–Nagumo, Allen–Cahn, Gray–Scott
Bounded	Field is physically stuck in a range	Cahn–Hilliard, Allen–Cahn (in [-1, 1])
Globally coupled	One point instantly affects all others	Navier–Stokes (pressure/Poisson)
Multi-field	Several quantities interact	shallow-water (3), wave / Gray–Scott / FHN (2)

---

2. What is a Neural Cellular Automaton?

A cellular automaton is a grid of cells where each cell updates itself using only what it can see in its immediate neighbourhood, with **the same rule everywhere**. Conway's Game of Life is the famous example.

A **Neural Cellular Automaton (NCA)** replaces the hand-written rule with a small neural network. Each cell:

1. **Perceives** — a 3×3 convolution gathers the cell's own value and its 8 neighbours.
2. **Processes** — a tiny per-cell MLP (built from 1×1 convolutions) decides what to do.
3. **Updates** — the cell adds a small increment to itself.

The same network is applied to every cell simultaneously, and the whole thing repeats.

Three properties follow directly from that design, and they explain almost everything in the results:

- **It is local.** One step moves information exactly one cell. To cross a 24-cell grid takes

24 steps. This is a good match for diffusion, and a bad match for anything with global coupling.

- **It is translation-equivariant and size-agnostic.** The same weights apply at every

position, and to any grid size. A model trained at 16×16 can be run at 48×48.

- **It is tiny.** A few thousand parameters, because the network is shared across all cells

rather than being a function of the whole grid.

The convolution uses **circular padding**, so the left edge wraps to the right and the top to the bottom. Periodic boundary conditions come for free — no boundary loss term needed.

Why "physics-informed"?

A plain NCA can output any increment it likes. Nothing stops the total amount of stuff in the grid from drifting up or down, which for a conservation law is simply wrong.

The physics-informed version fixes this **structurally**, not with a penalty term. Instead of predicting the change directly, it predicts a **flux** — how much material flows between neighbouring cells — and the change is computed as the divergence of that flux. Whatever leaves one cell arrives in another. Summed over a periodic grid the total change is exactly zero, by arithmetic, no matter what the network outputs.

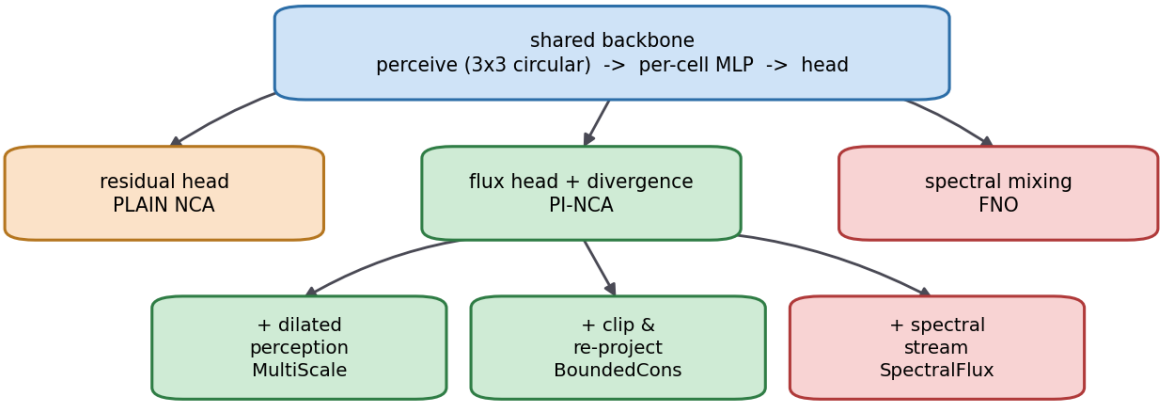
This is the key idea of the whole project: **build the physical law into the shape of the update, so it cannot be violated.**

---

3. The architecture family

Everything here is one shared backbone with a different ending. That is deliberate: it means a difference in results can be attributed to one component.

I. How the architectures relate - one backbone, different heads



All three hybrids are the conservative PI-NCA plus exactly ONE change. That is what makes the ablations clean: each hybrid isolates a single component - reach, boundedness, or global mixing.

Figure I — `docs/figures/arch/arch_family_tree.png`

Name in code	What it is	Params (heat)
`plain_nca`	NCA, residual head, no constraint	6 784
`pi_nca`	NCA, flux head — conserves mass	4 576
`multiscale_flux_nca`	PI-NCA + dilated multi-scale perception	5 520
`bounded_cons_nca`	PI-NCA + clip + mass re-projection	4 576
`spectral_flux_nca`	PI-NCA + a global spectral stream	134 225
`mc_flux_nca`	PI-NCA for multi-field states	10 992
`fno`	Fourier Neural Operator (not an NCA)	592 897

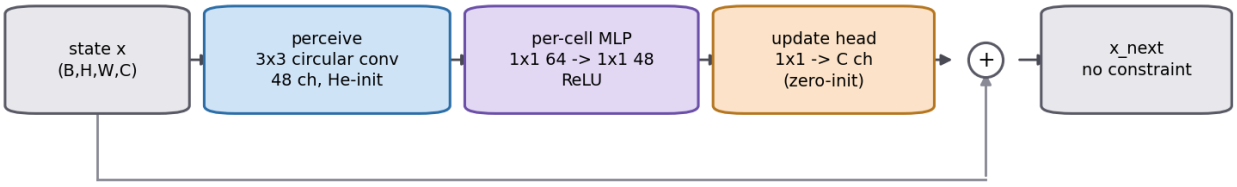
---

4. The architectures, one by one

A. Plain NCA — `models/nca.py`

The unconstrained baseline. Perceive, process, add the result to the state.

A. Plain NCA - models/nca.py



x(t+1) = x(t) + MLP(ReLU(perceive(x))) - a residual state increment.
Nothing constrains it, so total mass can drift.

Figure A — `docs/figures/arch/arch_plain_nca.png`

```
x_next = x + MLP(ReLU(perceive(x)))
```

Strengths. Cheap, simple, and completely unopinionated — it can represent any local update, including ones that create or destroy material. That freedom is exactly what makes it the best model on reaction problems.

Weaknesses. No conservation. On the heat equation its mass drifts by **33** while the flux version drifts by **0.0004**. Error also accumulates badly over long rollouts.

Best at: FitzHugh–Nagumo, Nagumo, wave — all non-conservative.

B. Conservative PI-NCA (DeepFluxNCA) — `models/flux_nca.py`

The core contribution. Same backbone, but the head outputs a two-channel flux field `(fx, fy)`, and the state update is the discrete divergence of that flux.

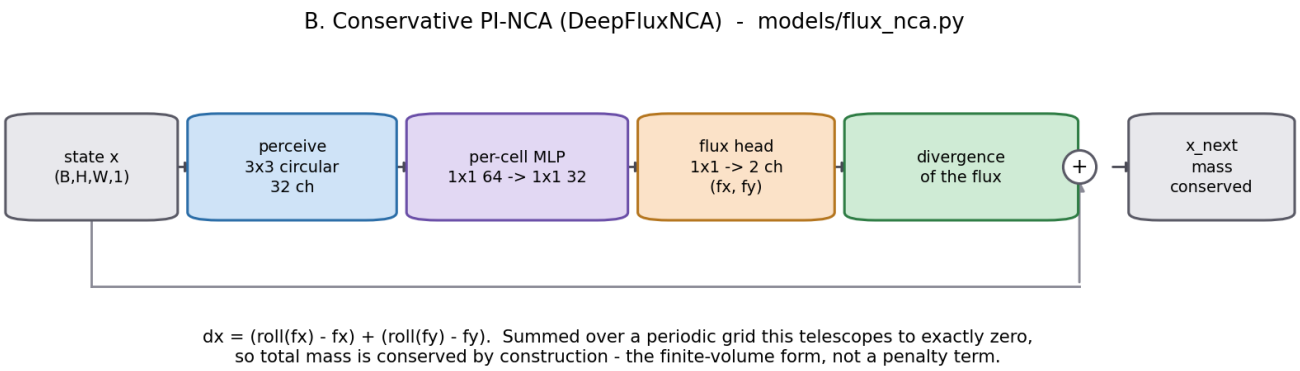


Figure B — `docs/figures/arch/arch_pi_nca.png`

```
dx = (roll(fx) - fx) + (roll(fy) - fy)
x_next = x + dx
```

Every term appears twice with opposite signs when you sum over a periodic grid, so the total cancels exactly. Mass is conserved to floating-point precision regardless of what the network learned. This is the finite-volume form used by classical conservative solvers.

Strengths. Exact conservation, and it is the *smallest* model in the study (4 576 parameters).

Weaknesses. Still strictly local. And conservation is a *prior* — when the real physics has source terms, enforcing it actively hurts (see ablation A4).

Best at: 3-D heat. Strong on advection–diffusion.

C. MultiScaleFluxNCA — `models/hybrids.py` ***(hybrid)***

The problem it solves: an NCA moves information one cell per step, which is too slow when the physics couples distant points.

The fix: perceive at three dilations at once — 1, 2 and 4 — and concatenate. The model now reaches ±4 cells per step instead of ±1, without an FFT and without many more weights.

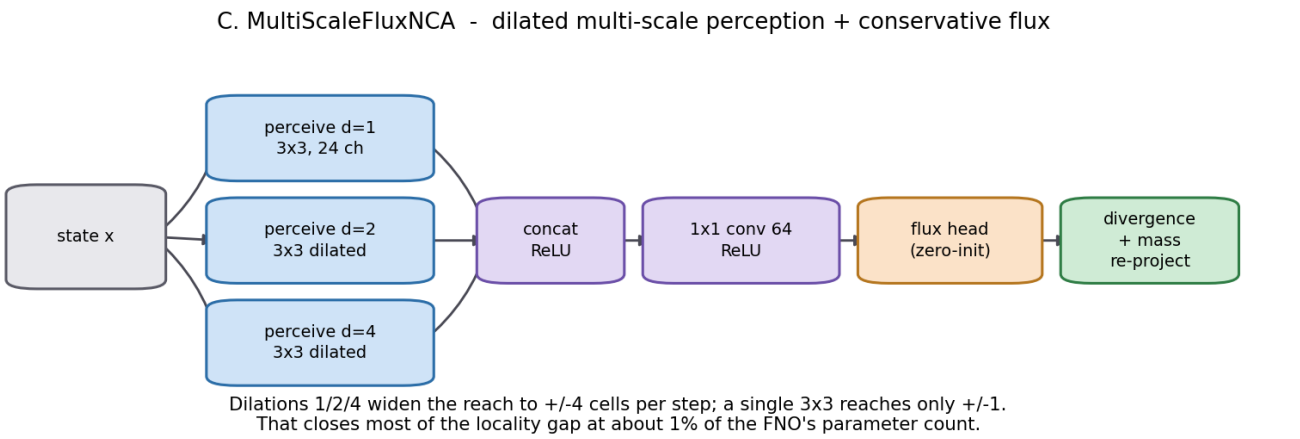


Figure C — `docs/figures/arch/arch_multiscale_flux_nca.png`

Result. Best 2-D heat model in the multi-seed run: **rel-L2 0.0183** at 5 520 parameters, beating the FNO's 0.0352 at 592 897 parameters. That is **1.9× better accuracy with 107× fewer parameters.**

Ablation A5 shows *how* you widen the reach matters. On Navier–Stokes a plain 5×5 stencil actually *destabilises* the model (1.486, worse than 3×3), while dilated multi-scale is best (0.388). Reach helps; reach the wrong way hurts.

Best at: heat (2-D). Best NCA on Navier–Stokes, though the FNO still wins there.

D. BoundedConsFluxNCA — `models/hybrids.py` *(hybrid)*

The problem it solves. On Cahn–Hilliard — a stiff equation whose field is physically stuck in [-1, 1] — every emulator blew up, reaching rel-L2 of 13–18 when simply predicting "nothing changes" would have scored 0.93.

Ablation A1 diagnosed it: unbounded network outputs drift outside the physical range and then explode. Clipping each step to [-1, 1] fixed the blow-up — a **24–27× improvement** — but *destroyed* conservation, because clipping arbitrarily adds and removes material (3.3e-5 → 7.6).

So stability and conservation were in direct conflict.

The fix. Record the total mass before the update. Do the conservative flux update. Clip. Then re-project the total mass back to the recorded value. Bounded *and* conserving.

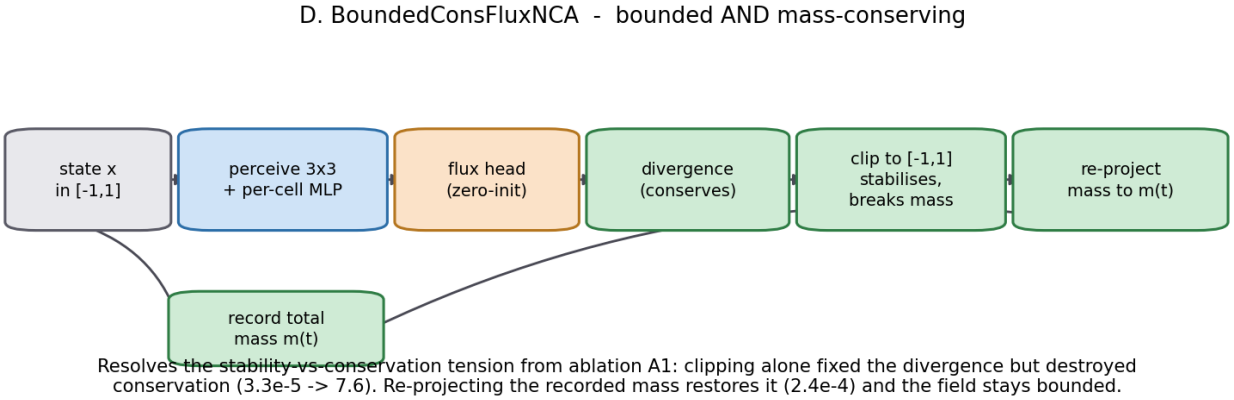


Figure D — `docs/figures/arch/arch_bounded_cons_nca.png`

Result. The Cahn–Hilliard winner: **rel-L2 0.603** with conservation error **2.4e-4** — both properties at once, where clipping alone gave you only one.

Best at: Cahn–Hilliard, and any stiff bounded field.

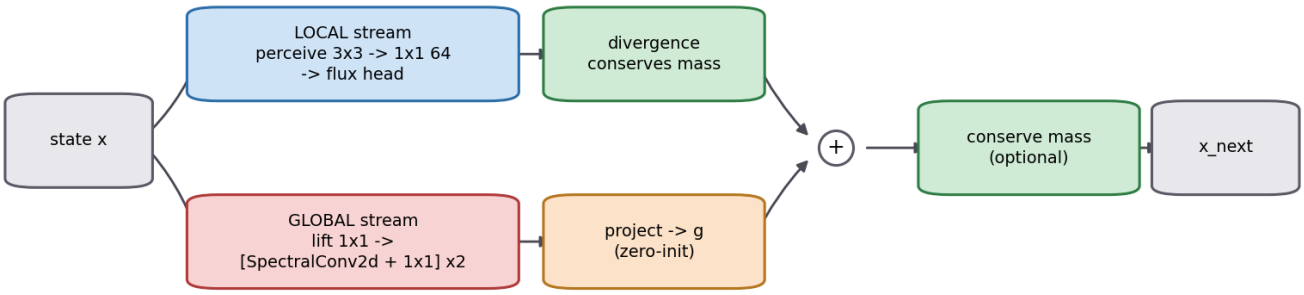
E. SpectralFluxNCA — `models/hybrids.py` *(hybrid)*

The idea. The central hypothesis of the project: take the FNO's global reach and the NCA's local conservation, and run them as two parallel streams.

- **Local stream:** perceive → MLP → flux head → divergence. Conserves mass.
- **Global stream:** lift → two spectral convolution layers → project. Sees everything at once.

The two are added, and the sum is optionally mass-projected.

E. SpectralFluxNCA - local conservation + global spectral correction



The central hybrid idea: the spectral stream supplies the global reach a local NCA lacks, while the flux-divergence stream keeps the combined update mass-conserving.

Figure E — `docs/figures/arch/arch_spectral_flux_nca.png`

Result — mixed, and worth being honest about. Under the single-seed better-start protocol it is the **best heat model in the study (rel-L2 0.0064)**. But in the 3-seed run it scored 0.0199 ± 0.014 — a standard deviation almost as large as the mean, against MultiScale's 0.0183 ± 0.0016. It can be the best, but it is **unstable across seeds**, and it costs 134 225 parameters — 24× MultiScale — for that.

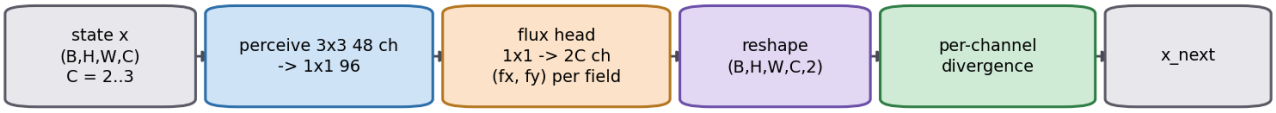
On Cahn–Hilliard it diverges completely (24.4), because it has no bounding mechanism.

Verdict: the hybrid hypothesis is only partly confirmed. Adding global mixing helps on heat, but cheaper dilated perception gets most of the same benefit far more reliably.

F. MultiChannelFluxNCA — `models/flux_nca.py`

For states with several interacting fields — shallow-water has 3 (height and two momenta), Gray–Scott and FitzHugh–Nagumo have 2. The head outputs `2C` channels: a separate flux pair per field, so **each field's total is conserved independently.**

F. MultiChannelFluxNCA - per-field conservation for multi-field states



Each field gets its own flux pair, so EVERY channel's total is conserved separately.
The right prior for shallow-water (mass + momentum); a deliberately wrong one for reaction systems, which have source terms.

Figure F — `docs/figures/arch/arch_mc_flux_nca.png`

Result — the sharpest illustration of the whole thesis. On shallow-water, where per-field conservation is physically correct, it conserves to **1.6e-4** and is competitive on accuracy (0.0311 vs FNO's 0.0259) with 54× fewer parameters.

On FitzHugh–Nagumo it conserves beautifully (**1.1e-6**) and is **useless** (rel-L2 0.993 — worse than doing nothing). FHN has source terms; its quantities are *not* conserved. The model enforces a law the physics does not obey.

Conserving the wrong thing perfectly is worse than not conserving at all.

G. Fourier Neural Operator (FNO) — `models/fno.py`

Not an NCA. The main competitor, and the standard method in the field.

Instead of looking at neighbours, it transforms the whole field to the frequency domain with an FFT, keeps only the lowest 8×8 frequency modes, multiplies them by learned weights, and transforms back. Every output point depends on

every input point — **global reach in one layer.**

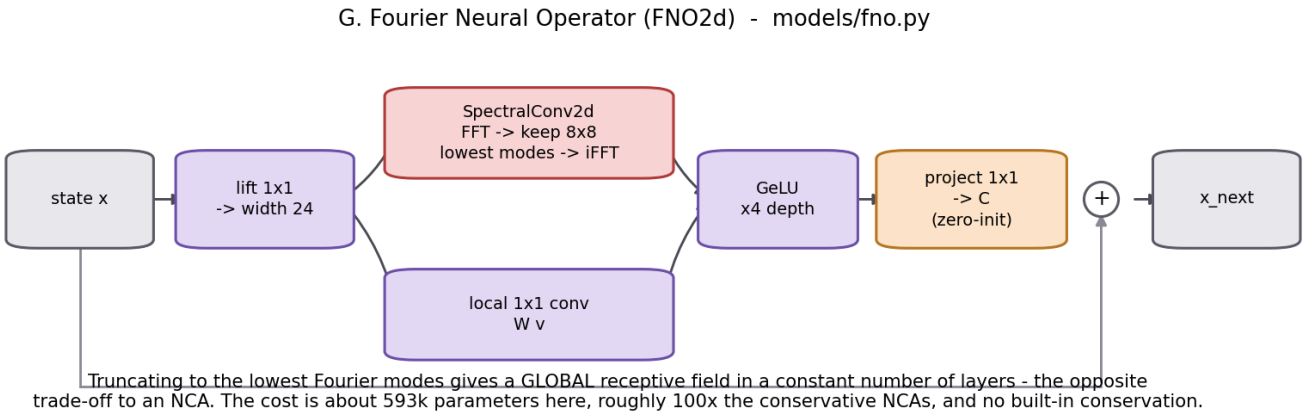


Figure G — `docs/figures/arch/arch_fno.png`

```
v_next = GeLU( W·v + iFFT( R ■ FFT(v) ) )
```

Strengths. Global coupling immediately, so it dominates Navier–Stokes. Keeping the whole spectrum means it also resolves sharp interfaces well — on heat its high-frequency error fraction is **0.004** versus the NCAs' 0.44–0.89.

Weaknesses. 592 897 parameters — roughly 100× the conservative NCAs. No conservation at all (heat mass drift: 21.3). Assumes a periodic regular grid.

Ablation A2 tested whether the FNO's advantage was architecture or just size: shrinking it to NCA budget (8 433 parameters) degraded it **3.4×**, from 0.035 to 0.119, losing to models at the same budget. **A meaningful part of the FNO's edge is parameter count.**

Best at: Navier–Stokes, Allen–Cahn, advection–diffusion, Nagumo.

H. The continuous baselines — PINN and DeepONet

These solve a genuinely different problem, so they are reported separately, not as head-to-head competitors.

PINN (`pinn_heat.py`) learns one continuous function `u(x, t)` for **one** initial condition, trained purely on the PDE residual with no data. Mesh-free and elegant, but it must be retrained from scratch for every new initial condition, and it struggles with high frequencies.

DeepONet (`deeponet_heat.py`) learns an *operator* — a mapping from an initial condition to the solution — so unlike a PINN it generalises across initial conditions.

Model	rel-L2 @ T	Params	Train time	Paradigm
DeepONet	**0.075 ± 0.009**	126 593	~9 s	operator, works across ICs
PINN	0.208 ± 0.018	14 209	~88 s	single problem, no training data

---

5. Hybrid results

The three hybrids, measured on the problems they were designed for. Multi-seed (3 seeds, mean ± std), grid 24, train 12 steps / evaluate 48 steps.

5.1 Heat — do the hybrids beat the baselines?

Source: `results/bench_heat_hybrid.md`

Model	rel-L2 ↓	MSE ↓	PSNR (dB) ↑	Conservation err ↓	Params ↓	Infer s/step ↓
multiscale_flux_nca	**0.0183 ± 0.0016**	**1.03e-3**	47.1	1.4e-3	5 520	7.5e-4
spectral_flux_nca	0.0199 ± 0.014	1.61e-3	**47.6**	1.5e-3	134 225	1.4e-3

Model	rel-L2 ↓	MSE ↓	PSNR (dB) ↑	Conservation err ↓	Params ↓	Infer s/step ↓
fno	0.0352 ± 0.0035	3.76e-3	41.5	21.3	592 897	3.1e-3
pi_nca	0.0438 ± 0.035	8.96e-3	41.4	**3.8e-4**	**4 576**	**5.7e-4**

Reading it. Both hybrids beat both baselines. MultiScale wins on accuracy, stability across seeds, parameters and speed simultaneously — it is the clear recommendation. SpectralFlux gets a marginally better PSNR but with a huge variance and 24× the parameters. Plain PI-NCA remains the conservation champion by 4×, and is the cheapest and fastest model.

5.2 Cahn–Hilliard — does the bounded-conserving hybrid work?

Source: `results/bench_cahn_hilliard_hybrid.md`. The identity baseline ("predict no change") scores 0.93; anything above that is worse than useless.

Model	rel-L2 ↓	PSNR (dB) ↑	Conservation err ↓	Params	Verdict
bounded_cons_nca	**0.603 ± 0.012**	**12.1**	2.4e-4	4 576	beats the identity floor
spectral_flux_nca	24.4 ± 20.0	-15.5	6.6e-4	134 225	diverges
multiscale_flux_nca	23.8 ± 8.0	-19.4	**4.0e-5**	5 520	diverges

Reading it. Bounding is what matters on this problem, and only one hybrid has it. The other two conserve mass immaculately while their solutions explode — a reminder that conservation is not stability. Note MultiScale's 4.0e-5 conservation error next to its rel-L2 of 23.8: perfectly conserved garbage.

5.3 The unified model

`MultiScaleFluxNCA` with `bounds=(-1,1)` set combines the heat-winning multi-scale reach with the CH-winning bounded-conserving update. Source: `results/bench_cahn_hilliard_unified.md`

Model	rel-L2 ↓	MSE ↓	Conservation err ↓	Params	Train wall (s)
bounded_cons_nca	**0.603 ± 0.012**	**0.246**	**2.4e-4**	**4 576**	62.9
bounded_multiscale_nca	0.633 ± 0.002	0.271	3.6e-4	5 520	**54.5**

Reading it. Once bounding is present, the extra multi-scale reach does **not** help on Cahn–Hilliard — it is 5% worse. Cahn–Hilliard is locally driven, so wider perception buys nothing. The unified model is more consistent across seeds (±0.002 vs ±0.012) and trains faster, but the simpler model is more accurate. **Combining two winning components does not automatically produce a better model.**

5.4 Hybrid scorecard

Hybrid	Problem it targeted	Did it work?
MultiScaleFluxNCA	NCA locality too slow	**Yes.** Best 2-D heat model, beats FNO at 1/107th the parameters.
BoundedConsFluxNCA	Stability vs conservation conflict	**Yes.** Only model that gets both; the CH winner.
SpectralFluxNCA	Locality, via global spectral mixing	**Partly.** Best single-seed heat score, but high variance, 24× params, and no bounding.

---

6. Comprehensive results

6.1 The regime map — the headline

Ten 2-D phenomena, single fixed seed 42 with the better-start protocol.

PDE	Character	Winner	rel-L2	Runner-up	Why
Heat	smooth, local	**spectral_flux_nca**	0.0064	fno 0.021	local + spectral

PDE	Character	Winner	rel-L2	Runner-up	Why
Advection–diffusion	linear transport	**fno**	0.0066	pi_nca 0.012	smooth global
Allen–Cahn	non-cons. phase separation	**fno**	0.0068	NCAs ~0.049	sharp interfaces
Wave	2nd-order hyperbolic	**plain_nca**	0.052	all ~0.05–0.057	effectively a tie
Shallow-water	conservative, multi-field	**mc_flux_nca**	0.016	fno 0.024	per-field conservation is correct
Nagumo	non-cons. bistable	**plain_nca**	0.073	fno 0.081	conservation prior hurts
FitzHugh–Nagumo	non-cons. reaction	**plain_nca**	0.125	fno 0.199	conservation prior hurts badly
Navier–Stokes	global coupling	**fno**	0.098	multiscale 0.285	locality fails
Cahn–Hilliard	stiff 4th-order, bounded	**bounded_cons_nca**	0.725	bounded_multiscale 0.790	bounding + conservation
Gray–Scott	reaction–diffusion patterns	**fno ≈ mc_flux**	0.674	mc_flux 0.692	hard; both poor

There is no universal winner. The structure of the PDE picks the architecture:

- **Conservative and smooth** → conservative NCAs (heat, advection–diffusion, shallow-water)
- **Globally coupled or sharp-featured** → FNO (Navier–Stokes, Allen–Cahn)
- **Non-conservative reaction** → unconstrained plain NCA (FHN, Nagumo)
- **Stiff and bounded** → bounded conserving NCA (Cahn–Hilliard)

Figure: `docs/figures/bench/bench_regime_map.png`

6.2 Per-PDE detail (multi-seed)

Heat — grid 24, eval 48, 3 seeds

Model	rel-L2 ↓	Conservation err ↓	Params	Infer s/step
multiscale_flux_nca	**0.0183 ± 0.0016**	1.4e-3	5 520	7.5e-4
spectral_flux_nca	0.0199 ± 0.014	1.5e-3	134 225	1.4e-3
fno	0.0352 ± 0.0035	21.3	592 897	4.4e-3
pi_nca	0.0438 ± 0.035	**3.8e-4**	4 576	5.7e-4
fno_small	0.119 ± 0.014	80.1	8 433	7.3e-4
plain_nca	0.234 ± 0.28	33.3	6 784	1.1e-3

Shallow-water — 3 fields, grid 24, eval 36, 2 seeds

Model	rel-L2 ↓	Conservation err ↓	Params
fno	**0.0259 ± 0.0015**	0.77	592 995
mc_flux_nca	0.0311 ± 0.0058	**1.6e-4**	10 992
plain_nca	0.0399 ± 0.0028	4.81	7 744

FitzHugh–Nagumo — 2 fields, grid 24, eval 48, 2 seeds

Model	rel-L2 ↓	Conservation err ↓	Params
plain_nca	**0.154 ± 0.0099**	118.1	7 264
fno	0.452 ± 0.020	75.2	592 946
mc_flux_nca	0.993 ± 0.0078	**1.1e-6**	10 464

Navier–Stokes — grid 24, eval 48, 2 seeds

Model	rel-L2 ↓	Conservation err ↓	Params
fno	**0.145 ± 0.029**	1.93	592 897

Model	rel-L2 ↓	Conservation err ↓	Params
multiscale_flux_nca	0.284 ± 0.030	1.0e-4	5 520
plain_nca	0.528 ± 0.050	7.28	6 784
pi_nca	0.676 ± 0.17	2.1e-5	4 576

Nagumo — grid 24, eval 48, 2 seeds

Model	rel-L2 ↓	Params
fno	**0.118 ± 0.009**	592 897
plain_nca	0.119 ± 0.006	6 784
pi_nca	0.379 ± 0.001	4 576
multiscale_flux_nca	0.379 ± 0.001	5 520

Full 20-metric tables for every phenomenon: `results/bench_<pde>_full.md`.

6.3 Ablations — which component actually matters

#	Question	Setup	Result
A1	Does bounding fix stiff PDEs?	CH, clip to [-1,1]	plain 12.9 → 0.54, pi_nca 16.5 → 0.60 (**24–27×**), but clipping breaks conservation (→7.6)
A2	Is the FNO's edge architecture or size?	FNO at NCA budget (8.4k)	0.035 → 0.119 (**3.4× worse**) — much of the edge was parameter count
A3	Does training horizon matter?	CH, train 12/24/48	0.633 → 0.619 → **0.511**; conservation 3.2e-4 → 6.8e-5
A4	Does conservation help?	Same backbone, flux vs residual head	Heat **0.104 vs 0.353** (helps 3.4×); Nagumo **0.379 vs 0.123** (hurts 3.1×)
A5	Does wider perception help?	3×3 / 5×5 / dilated (1,2,4)	Heat 0.104 / 0.046 / **0.024**; NS 0.576 / 1.486 / **0.388**
A6	Does multi-step BPTT matter?	train 1/4/12 steps	Heat 0.025 → **0.021** (16%); NS 1.562 → **0.284** (**5.5×**)

A4 is the cleanest result in the study. Identical backbone, identical width, only the head differs. Conservation helps by 3.4× on a conservative PDE and hurts by 3.1× on a non-conservative one. The inductive bias is not universally good — it is *correct or incorrect* for the physics.

A6 says the training horizon matters most where the dynamics are hardest. On stable heat it barely registers; on unstable Navier–Stokes, single-step training diverges (1.562) while 12-step training controls the rollout (0.284).

Figures: `docs/figures/bench/bench_ablation_A4.png`, `bench_ablation_A5.png`

6.4 Efficiency — what accuracy costs

Cost of accuracy on heat, measured as rel-L2 × parameters (lower is better):

Model	rel-L2 × params	Relative
multiscale_flux_nca	**101**	1×
pi_nca	200	2×
fno_small	1 003	9.9×
fno	20 875	**207×**

On the problems where a conservative NCA is the right tool, it is not marginally cheaper — it is two orders of magnitude cheaper.

Figure: `docs/figures/bench/bench_accuracy_vs_cost.png`

6.5 3-D — the conclusions hold

Full 3-D pipeline at 16^3, single seed 42.

PDE	Character	Winner	rel-L2	Conservation (NCA vs plain·FNO)
Heat	local diffusion	**pi_nca**	0.047	1.8e-3 vs 50 · 145
Advection–diffusion	smooth transport	**fno**	0.008	1.4e-3 vs 28 · 4
Allen–Cahn	sharp interfaces	**fno**	0.012	1.2e-4 vs 3.5 · 14
Nagumo	non-cons. bistable	**plain_nca**	0.041	conserving NCAs worst (0.198)
FitzHugh–Nagumo	non-cons. reaction	**fno**	0.163	mc_flux 0.99 — conserves, useless
Gray–Scott	reaction patterns	plain ≈ fno	0.42	mc_flux 1.09, diverges

Every 2-D conclusion reproduces in 3-D. Local diffusion → conservative NCA wins at ~3k parameters against the FNO's 747k. Non-conservative → the conservation prior hurts, again. The thesis is dimension-independent.

Figure: `docs/figures/bench/bench_accuracy_3d.png`

6.6 Resolution transfer

Train at one grid size, evaluate at another. Both families are size-agnostic in principle; in practice transfer is regime-dependent.

PDE	Finding
Heat	NCAs transfer coarse→fine well (16→48: 0.029) but poorly fine→coarse (48→16: 0.349). The FNO is accurate only near its training resolution (16→48: 0.504) — **not** resolution-invariant here.
Allen–Cahn	Both robust. MultiScale ~0.05 flat; FNO 0.012–0.026.
Navier–Stokes	FNO **genuinely resolution-invariant** (~0.18–0.24 across all pairs). The NCA fails to transfer and diverges when trained at 48^2 (1.25–2.39).

Figures: `docs/figures/bench/bench_resolution_heat.png`, `bench_resolution_allen_cahn.png`, `bench_resolution_navier_stokes.png`

---

7. What to take away

1. **No universal winner.** Architecture should be chosen from the PDE's structure — conservation, boundedness, and how far information travels per step.
2. **Structural constraints beat penalty terms.** Flux-divergence conserves mass to 1e-4 or better *by construction*. A loss term only encourages it.
3. **The right prior is worth ~100× the parameters.** On heat, 5 520 parameters beat 592 897.
4. **The wrong prior is worse than none.** Per-field conservation on FitzHugh–Nagumo conserves to 1e-6 and produces a useless model (0.993).
5. **Conservation is not stability.** On Cahn–Hilliard, models conserved mass perfectly while diverging to rel-L2 24. Bounding was the missing ingredient.
6. **Hybrids work when they target a specific measured failure.** MultiScale and BoundedCons each fixed a diagnosed problem and won their regime. SpectralFlux was the most ambitious and delivered least reliably. Stacking two winners (bounded + multi-scale) made things slightly *worse*.

---

8. Figure index

All paths relative to the repository root.

Architecture diagrams — `docs/figures/arch/`

File	Shows
`arch_family_tree.png`	How all architectures relate to one backbone
`arch_plain_nca.png`	Plain NCA
`arch_pi_nca.png`	Conservative PI-NCA (flux divergence)
`arch_multiscale_flux_nca.png`	MultiScaleFluxNCA hybrid
`arch_bounded_cons_nca.png`	BoundedConsFluxNCA hybrid
`arch_spectral_flux_nca.png`	SpectralFluxNCA hybrid
`arch_mc_flux_nca.png`	MultiChannelFluxNCA
`arch_fno.png`	Fourier Neural Operator
`arch_training_pipeline.png`	Shared training pipeline

Regenerate with `python -m pinca_jax.arch_figs`.

Benchmark plots — `docs/figures/bench/`

File	Shows
`bench_regime_map.png`	Which architecture wins which PDE
`bench_accuracy_2d.png`	rel-L2, all models, all 2-D phenomena
`bench_accuracy_3d.png`	The 3-D suite
`bench_conservation_2d.png`	Mass drift — where the flux head earns its keep
`bench_accuracy_vs_cost.png`	Accuracy vs parameter count
`bench_error_growth.png`	Error growth from T/4 to T
`bench_psnr_2d.png`	PSNR
`bench_train_time.png`, `bench_throughput.png`	Training cost, inference speed
`bench_ablation_A4.png`, `bench_ablation_A5.png`	Conservation on/off; perception size
`bench_resolution_*.png`	Train-grid × eval-grid transfer

Regenerate with `python -m pinca_jax.plots`.

Simulation figures — `docs/figures/`

File pattern	Shows
`<pde>_comparison.png`	Analytic vs model vs error, 2-D, at key timesteps
`<pde>_3d_comparison.png`	Same in 3-D, mid-depth slice
`<pde>_3d_volume.png`	True 3-D volume render

For `<pde>` in: heat, allen_cahn, nagumo, adv_diff, gray_scott, shallow_water, fitzhugh_nagumo, wave, cahn_hilliard, navier_stokes (2-D); heat, adv_diff, allen_cahn, nagumo, gray_scott, fitzhugh_nagumo (3-D).

---

9. Source files

Component	Path
Plain NCA	`src/pinca_jax/models/nca.py`
Conservative PI-NCA, multi-channel	`src/pinca_jax/models/flux_nca.py`
All three hybrids	`src/pinca_jax/models/hybrids.py`

Component	Path
FNO	`src/pinca_jax/models/fno.py`
Ablation backbone	`src/pinca_jax/models/ablation_nca.py`
Conservation operators	`src/pinca_jax/physics.py`
PDE solvers (teachers)	`src/pinca_jax/equations/pdes.py`
Training / evaluation harness	`src/pinca_jax/harness.py`
3-D counterparts	`src/pinca_jax/*3d.py`
Benchmark drivers	`src/pinca_jax/bench.py`, `bench_all.py`, `bench3d.py`
Correctness gate	`tests/` (56 tests)

Reproduce: `python -m pytest tests/ -q`, then `bash run_gpu.sh` (see `docs/gpu_runbook.md`).